

Tenho um app que está em desenvolvimento com MAUI 10 e que persegue ser MD3 compliant, usando DevExpress MAUI

Aqui está o plano que montei. Me avise se precisar de alguma alteração antes de eu começar a pesquisar.

Otimização de Design MAUI MD3

Pesquisar sites

(1) Pesquise no Google Workspace pela pasta 'MyVocaList/SourceCode-20260601' e extraia detalhes do código-fonte para analisar o design atual, a estrutura da...

Analisar os resultados

Criar relatório

Só mais uns minutinhos

Editar o plano de pesquisa

Este é o plano atual. Que alterações quer fazer?

(1) Pesquise no Google Workspace pela pasta

'MyVocaList/SourceCode-20260601'

e extraia detalhes do código-fonte para analisar o design atual, a estrutura da interface e a implementação dos componentes do DevExpress MAUI.

(2) Pesquise na web pelas melhores práticas e diretrizes do Material

Design 3 (MD3) em

<https://m3.material.io/> com foco em

esquemas de cores vibrantes, temas claros e formas de evitar layouts monótonos.

(3) Encontre alternativas modernas de sistemas de design ao MD3 (como Fluent Design ou Cupertino) que sejam suportadas pelo .NET MAUI 10 e que ofereçam interfaces atrativas e performáticas para Android, iOS e Windows.

(4) Procure por atualizações recentes, soluções de contorno ou bibliotecas alternativas de UI para MAUI 10 que resolvam a falta de suporte do DevExpress MAUI no Windows sem reintroduzir problemas de concorrência de thread de renderização.

(5) Pesquise sobre o uso da ferramenta 'Stitch', servidores MCP (Model Context Protocol) e a integração com o 'Claude Code' para automatizar processos de criação de UI e garantir a conformidade de design (Stitch Planned Compliant).

(6) Avalie os dados coletados nos passos anteriores para comparar o uso contínuo do MD3 com DevExpress versus a adoção de um novo framework visual que atenda nativamente aos 3 sistemas

operacionais de forma performática.
(7) Sintetize as informações em um relatório contendo recomendações práticas de design para tornar o aplicativo menos escuro e monótono, juntamente com o fluxo de integração do Stitch e MCP para o desenvolvimento futuro.

Considere que o APP é planejado para gestão de eventos de Karaoke/Bandokê, e que deseja-se ter um design que remeta a esta temática.

Aqui está o plano atualizado. Fique à vontade para mudar qualquer coisa.

Design App Karaoke: MD3
vs. Alternativas

Pesquisar sites

(1) Pesquise no Google
Workspace pela pasta
'MyVocaList/SourceCode-
20260601' e extraia detalhes
do código-fonte para analisar o
design atual, a estrutura da...

Analisar os resultados

Criar relatório

Só mais uns minutinhos

Iniciar pesquisa

Terminei a pesquisa. Fique à
vontade para me fazer perguntas
complementares ou pedir
alterações.

Design App Karaoke: MD3
vs. Alternativas

1 de jun., 14:39

A adoção do UraniumUI,
apesar de parecer fazer
sentido dado ter
conformidade com MD3
nos OSs Androic

Compreendo perfeitamente o
dilema. O seu ponto é
extremamente válido: o objetivo do
projeto é desenvolver um aplicativo
de gestão de Karaokê (focado na
regra de negócio e na experiência
do usuário) e não criar e manter
uma biblioteca de componentes de
interface do zero. Se o UraniumUI
não possui a vasta gama de
componentes prontos (como
DataGrids avançados, Schedulers e
painéis complexos) que o
DevExpress oferece, o esforço de
criar esses elementos manualmente

anula o benefício de ter um framework gratuito.

Dado que o requisito de suportar Windows e plataformas móveis é central, e que você está aberto a abandonar o Material Design 3 (MD3) estrito em favor de frameworks que não exijam "reinventar a roda", aqui está a avaliação de três rotas arquiteturais altamente viáveis e eficientes:

1. A Rota de Componentes Ricos em XAML: Syncfusion (Licença Community)

Se você deseja manter a aplicação escrita em C# e XAML nativo (semelhante ao que já faz com o DevExpress), a alternativa mais direta e robusta do mercado é o Syncfusion Essential Studio para .NET MAUI.

Suporte Integral: Diferente do DevExpress, a Syncfusion oferece suporte nativo e testado para Android, iOS, macOS e Windows (via WinUI 3) a partir do mesmo código-

fonte. O suporte para o.NET 10 é classificado como "Day-1".

Acervo de Componentes: Eles possuem mais de 100 componentes prontos, incluindo substitutos diretos e mais poderosos para o que você usa hoje no DevExpress, como DataGrids virtuais, Schedulers, painéis retráteis (BottomSheets) e visualizadores de PDF.

O Fator Custo: Embora seja uma ferramenta corporativa paga, a Syncfusion possui a Community License, que fornece acesso 100% gratuito a todos os controles para desenvolvedores individuais ou pequenas empresas (geralmente com faturamento inferior a 1 milhão de dólares anuais).

Design e Temas: Você não fica preso ao MD3 estrito. Eles suportam temas customizados e efeitos modernos (como o

Liquid Glass UI, excelente para a estética Neon do seu app).

2. A Rota Web Híbrida: MAUI Blazor Hybrid (Bootstrap / Tailwind)

Você mencionou a possibilidade de adotar o "Bootstrap" ou similares. No ecossistema MAUI, a maneira oficial, performática e recomendada de utilizar bibliotecas de design web (como Bootstrap, Tailwind CSS, Flowbite ou Radzen) é alterando a camada de apresentação para .NET MAUI Blazor Hybrid.

Como funciona: O aplicativo continua sendo 100% nativo e compilado instalável nas lojas (iOS, Android, Windows), mas a interface visual é renderizada usando componentes Web dentro de um controle embutido de altíssimo desempenho, rodando no mesmo processo do C# local (sem necessidade

de servidor web ou WebAssembly).

Não reinventar a roda: Esta rota abre as portas para o maior ecossistema de componentes prontos do mundo (a web). Você pode usar milhares de templates e bibliotecas baseadas em Bootstrap ou Tailwind prontas para uso, reduzindo o tempo de design de interface a uma fração do que seria no XAML puro. O.NET 10 trouxe melhorias consideráveis de performance para essa arquitetura.

3. A Rota MAUI Nativo Puro (Estilo Customizado sem MD3 Estrito)

Se preferir não adicionar pacotes pesados de terceiros, você pode usar puramente as bibliotecas padrão de UI do .NET MAUI (`Grid` , `CollectionView` , `Button` , etc.) e abandonar o dogma do MD3.

Global Styles: O MAUI permite que você construa dicionários de recursos

(`ResourceDictionary`) globais no arquivo `App.xaml` . Você pode definir a sua paleta "Neon Purple/Pink" festiva nesses recursos, e todos os botões e painéis herdarão o design instantaneamente, conferindo uma aparência única e unificada ao app.

Integração de Tokens:

Ferramentas da comunidade, como o

`TailwindXamlThemeConverter`, permitem converter as paletas de cores e design tokens diretamente para variáveis XAML do MAUI. Isso significa que você pode desenhar no Figma ou usar ferramentas como o Google Stitch para projetar o visual, e aplicar os estilos aos controles nativos do MAUI, alcançando o visual desejado sem construir novas lógicas de componentes.

Síntese e Recomendação

O seu sacrifício inicial (abandonar o Windows para ter componentes prontos com o DevExpress) não é mais necessário.

Se o seu foco principal for a produtividade na criação de formulários e listas complexas sem sair da linguagem XAML, a migração do DevExpress para o Syncfusion (via Community License) é a escolha mais lógica. Eles resolvem o problema de ausência de componentes do UraniumUI e suportam o Windows nativamente.

Se você prefere focar em um design ultra moderno e flexível com a vasta gama de componentes prontos da web (como o Bootstrap), converter a camada de UI para MAUI Blazor Hybrid

definitivamente. Também haverá um site comercial, com foco em comunicações e engajamento da comunidade. E, |

Diante da nova premissa de desenvolver um site comercial focado em engajamento e, futuramente, recursos de nuvem integrados que exijam soluções web executadas diretamente no navegador, a escolha arquitetural que oferece o menor custo, o menor esforço de migração e a maior homogeneidade visual é, de forma inequívoca, o .NET MAUI Blazor Hybrid utilizando uma Razor Class Library (RCL).

Esta abordagem resolve perfeitamente o dilema da escassez de componentes, a falta de suporte ao Windows do DevExpress e a necessidade de uma interface idêntica entre o App e a Web.

Por que esta é a escolha mais óbvia e eficiente?

1. Compartilhamento Real de UI (Escreva uma vez, rode no App e na Web)

O maior benefício de adotar o Blazor Hybrid no.NET 10 é a capacidade de criar uma Razor Class Library (RCL).

Você escreve as telas do aplicativo (HTML, CSS e componentes C# Razor) dentro desta biblioteca compartilhada.

O seu App Multiplataforma (.NET MAUI) consome essa RCL e renderiza a interface nativamente no Android, iOS e Windows através de um controle WebView de altíssimo desempenho, rodando no mesmo processo do C# (sem emulação ou latência de rede).

O seu Site Comercial/Comunidade (um Blazor Web App na nuvem) consome exatamente a mesma RCL, exibindo a mesma interface diretamente nos navegadores dos usuários.

Resultado: Se você alterar a cor de um botão de Karaoke ou a disposição da lista de músicas, a mudança se refletirá instantaneamente no app de celular, no sistema desktop do Windows e no site web.

2. Fim do Dilema de Componentes ("Não reinventar a roda")

Ao migrar a interface para a Web (HTML/CSS), você deixa de depender de bibliotecas XAML limitadas ou caras. Você ganha acesso ao maior ecossistema de componentes prontos do mundo:

É possível adotar frameworks CSS extremamente populares como o Tailwind CSS ou Bootstrap, combinados com bibliotecas de componentes prontas e responsivas (como MudBlazor, Radzen ou Flowbite).

Essas bibliotecas oferecem DataGrids virtuais para

milhares de músicas, modais, BottomSheets e menus colapsáveis perfeitamente responsivos para desktop e mobile.

3. Custo de Migração de Código Quase Zero no Back-End

Como o seu MVP foi estruturado usando padrões de classe empresarial, MVVM e DDD em C#, a transição para Blazor Hybrid preserva quase todo o seu investimento em código:

O Blazor Hybrid roda nativamente na máquina física (não usa WebAssembly no app local). Isso significa que seu código C# de acesso ao SQLite, regras de negócio em DDD e serviços de comunicação continuam rodando exatamente como estão.

A única alteração será a substituição das Views XAML por componentes Razor

(HTML/C#). Os seus ViewModels existentes podem ser facilmente adaptados para fornecer os dados às páginas Razor via injeção de dependência padrão do.NET.

4. Sinergia Perfeita com o Google Stitch e `DESIGN.md`

Como a sua UI passará a ser baseada em Tailwind CSS ou CSS nativo, o fluxo com o Claude Code e o Google Stitch atinge o seu potencial máximo :

O Stitch gera o arquivo `DESIGN.md` com a sua paleta Neon e regras visuais de Karaoke.

Através do CLI do Stitch, você exporta esses tokens de design diretamente para um arquivo de configuração do Tailwind (`tailwind.config.js`) ou CSS variáveis.

O Claude Code lerá este arquivo e criará componentes Razor com o estilo visual rigorosamente idêntico ao prototipado no Stitch, garantindo o padrão Stitch Planned Compliant de forma automatizada tanto para o site quanto para o aplicativo.

5. Desempenho e Recursos do.NET 10

O.NET 10 traz melhorias de desempenho muito expressivas para o ecossistema híbrido, incluindo otimizações de memória e tempo de inicialização. Além disso, a nova capacidade de interceptar requisições web no `BlazorWebView` permite que o app mobile manipule dados locais (como arquivos de áudio de Karaoke ou o banco SQLite local) de forma extremamente segura e invisível para o motor web.

Conclusão e Próximo Passo

Substituir o DevExpress MAUI pelo MAUI Blazor Hybrid é a decisão que remove todas as barreiras tecnológicas do seu projeto:

1. Viabiliza o app no Windows, Android e iOS com uma única base de código.
2. Permite que o futuro site web reaproveite 100% da interface do aplicativo.
3. Acaba com a necessidade de criar componentes visuais complexos do zero, utilizando o vasto ecossistema de componentes web responsivos.